

Development Plan FitFolio

Team #1, Burn n' Earn
Davis Lenover
Preston Grewal
Angad Chhabra
Farid Bastoros
Zaid Alsaadi

Table 1: Revision History

Date	Developer(s)	Change
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

[What copyright license is your team adopting. Point to the license in your repo. —SS]

4 Team Meeting Plan

[This section is for meetings between the team members, and meetings between the team and their supervisor/advisor (as appropriate). —SS]

[How often will the team meet? where? when? —SS]

[If the meeting is a physical location (not virtual), out of an abundance of caution for safety reasons you shouldn’t put the location online —SS]

[To help schedule meetings, you can use the lecture/tutorial time when we do not have anything else scheduled. —SS]

[How often will you meet with your supervisor/advisor? when? where? —SS]

[Will meetings be virtual? At least some meetings should likely be in-person. —SS]

[How will the meetings be structured? There should be a chair for all meetings. There should be an agenda for all meetings. —SS]

5 Team Communication Plan

[Issues on GitHub should be part of your communication plan. —SS]

6 Team Member Roles

[You should identify the types of roles you anticipate, like notetaker, leader, meeting chair, reviewer. Assigning specific people to those roles is not necessary at this stage. In a student team the role of the individuals will likely change throughout the year. —SS]

7 Workflow Plan

This section describes how the team will use Git and GitHub to collaborate on both source code and documentation, how work will be tracked through issues, and how Continuous Integration and Continuous Deployment (CI/CD) will keep the project stable. It is written to serve as a guide for current and future contributors. The repository is available at [BurnNEarn GitHub Repository](#).

7.1 Branches

The repository will have exactly one permanent branch:

- **main**: the protected, stable branch representing the latest reviewed state of the project. Direct pushes are not permitted. All changes must arrive through a pull request that has been approved by at least one other team member and has passed all CI checks.

All other branches are short-lived and must follow the naming convention `<type>/<name>-<description>`, where **type** is one of the categories in Table 2, **name** is the author's first name in lowercase, and **description** is a brief summary of the work in two to four words, separated by hyphens.

Table 2: Branch Types and Naming Examples

Type	Used For	Example
docs	Documentation sections	docs/farid-dev-plan-workflow
feature	New app functionality	feature/davis-step-sync
fix	Bug fixes	fix/angad-portfolio-rounding
chore	CI, configuration, and repository maintenance	chore/preston-latex-build

Branches are not shared between members; each member creates their own branch for each unit of work. For documentation, each member creates one branch per deliverable covering the sections assigned to them. For code, each feature or fix gets its own branch. Once a branch's pull request is merged into **main**, the branch is deleted. Before opening a pull request, the author updates their branch with the latest changes from **main** and resolves any merge conflicts locally, so that conflicts are never resolved during review.

7.2 Commits

Commits should be small and focused on a single change, rather than combining unrelated edits. Commit messages should begin with a short summary in the imperative mood (e.g., “Add step count permission request”) and reference the related issue number where applicable (e.g., **#14**), which links the commit to that issue on GitHub. Issues are closed through the pull request rather than individual commits, as described in Section 7.3.

7.3 Pull Requests

When the work on a branch is complete, the author follows these steps:

1. Open a pull request into **main** with a title summarizing the change and a description of what was changed and why. The description must link the related issue using a closing keyword (e.g., **Closes #14**), which automatically closes the issue when the pull request is merged.
2. Request at least one reviewer. The reviewer should be a member working in a related area, or the whole team if the change affects everyone, such as shared document sections or the overall architecture.
3. Address the reviewer’s comments, which are left directly on the relevant lines of code or documentation.
4. Once the pull request has one approval and all CI checks pass, the author merges it using a *merge commit*, which preserves every individual commit from the branch on **main**. This keeps a complete record of each member’s contributions throughout the project. The author then deletes the branch.

Reviewers are expected to respond to a review request within 48 hours, or within 24 hours during the final three days before a deadline. For documentation, reviewers also check the changes against the deliverable’s checklist and rubric before approving.

7.4 Tags

Deliverables are marked directly from the **main** branch, so a Git tag will be created on **main** at each deliverable’s deadline to record exactly what was marked. Document deadlines use the format **<document>-<revision>** (e.g., **dev-plan-rev0**, **srs-rev0**, **design-rev-1**), and demonstrations use the format **<demo>-demo** (e.g., **poc-demo**, **rev0-demo**, **final-demo**). The team member who merges the final pull request before a deadline is responsible for creating its tag. Tags allow the team to return to the exact state of the project at each deadline, and to compare revisions when addressing feedback.

7.5 Issue Management

GitHub Issues will serve as the team’s ticketing system, so that all work is visible, assigned, and traceable.

- **Issue size:** Each issue should represent work that can be completed in a single sitting of roughly two to three hours. Larger tasks are split into multiple issues. For documentation, every section of each document has its own issue, titled with its deliverable and section (e.g., [Development Plan] 7 Workflow Plan).
- **Ownership:** Every task issue is assigned to one owner, who is responsible for completing it and moving it through the project board described in Section 8.1.
- **Milestones:** Each deliverable is a milestone whose due date matches the deadline in Table 6. Every issue related to a deliverable is added to its milestone, so the team can see how much work remains before each deadline.
- **Meetings and lectures:** An issue is created for every team meeting, supervisor meeting, and lecture, and is tracked in its own column on the project board. At least 24 hours before a meeting, the agenda and any questions are added to the issue. Afterwards, notes and action items are recorded, each action item becomes its own issue, and the meeting issue is closed. Lecture issues record the topic, date and time, and attendance.

The repository includes the issue templates listed in Table 3, so that every issue contains consistent information.

Table 3: Issue Templates

Template	Contents
Documentation Task	Deliverable, section, and the checklist items the section must satisfy
Feature	Description of the functionality and acceptance criteria
Bug Report	Steps to reproduce, expected and actual behaviour, and device and operating system
Meeting	Meeting type, date and time, attendance, agenda, questions, notes, and action items
Lecture	Topic, date and time, and attendance

Issues are classified using the labels in Table 4. Every issue that belongs to a deliverable receives that deliverable’s label, and every issue receives one type label and, where relevant, one area label. Together, these allow the team to filter issues by deliverable, by kind of work, or by part of the app.

Table 4: Issue Labels

Category	Label	Meaning
Deliverable	Problem Statement and Goals	Problem Statement and Goals document
	Development Plan	Development Plan and Proof of Concept Plan
	Requirements	Software Requirements Specification
	Hazard Analysis	Hazard Analysis document
	V&V Plan	Verification and Validation Plan
	Design Document	Design Document (all revisions)
	V&V Report	Verification and Validation Report
	Demo	Proof of Concept, Revision 0, Final, and EXPO demonstrations
Type	documentation	Writing or revising a document section
	reflection	Reflection components of a deliverable
	feature	New functionality
	enhancement	Improvement to existing functionality
	bug	A defect that must be fixed
	meeting	Team or supervisor meeting
Area	lecture	Course lecture
	frontend	User interface of the mobile app
	backend	Server, database, and accounts
	health-data	Fitness tracking and health platform integration
	market-engine	Asset prices, portfolios, and market scenarios
	social	Social feed and sharing

7.6 Continuous Integration and Continuous Deployment

GitHub Actions will run automated checks on the repository, so that errors are caught when a change is proposed rather than after it reaches `main`:

- **Documentation:** On every pull request that changes the `docs` folder, all $\text{\LaTeX}$ documents are compiled. The check fails if any document does not build, and the generated PDFs are saved as downloadable artifacts of the workflow run, linked from the pull request, so reviewers can read the rendered changes.
- **Code:** Once development begins, every pull request and every push to `main` runs a linter and the automated test suite in the `test` folder. Branch protection prevents a pull request from being merged if any check fails.
- **Deployment:** Once the core features are stable, targeted for Revision 0, every merge into `main` will automatically produce an internal test build of

the mobile app, so the team and supervisor can always install the latest version on their phones.

If a check fails, the pull request's author is responsible for fixing it before requesting another review.

8 Project Decomposition and Scheduling

This section describes how the team will use GitHub Projects to track work, how each deliverable will be broken down into manageable tasks, and how the project will be scheduled around both course deadlines and earlier team deadlines.

8.1 GitHub Projects

The team will use a GitHub Project board, *BurnNEarn-TrackingBoard*, as the central place to track all issues. It is available at [FitFolio GitHub Project](#). Every task issue moves through the first four columns in Table 5 in order, while meeting and lecture issues are kept in their own columns so they do not clutter the task workflow.

Table 5: Project Tracking Board Columns

Column	Description
Backlog	Tasks that have been identified but not yet started. New issues are added here automatically.
In Progress	Tasks being actively worked on. The owner moves an issue here when they create its branch.
In Review	Tasks with an open pull request awaiting review. The owner moves an issue here when they open the pull request.
Done	Tasks whose pull request has been approved and merged. Issues move here automatically when they are closed.
Meetings	Team and supervisor meetings, with their agendas, notes, and action items.
Lectures	Course lectures, with their topic, date and time, and attendance.

The board can be filtered by milestone, assignee, or label. Filtering by the current milestone shows how much work remains before the next deadline, and filtering by assignee shows each member's workload, which helps the team balance tasks during weekly meetings.

8.2 Task Decomposition

At the start of each deliverable, the team breaks the work into issues and adds them to that deliverable's milestone. Work is decomposed differently depending on its type:

- **Documentation** is broken down by section, with one issue per section titled with its deliverable and section number (e.g., [Development Plan] 8 Project Decomposition and Scheduling). Each section can then be written, reviewed, and merged independently.
- **Software** is broken down by feature, and each feature is split further into issues that fit within a single sitting and can be tested on their own.

Issues are assigned during the weekly team meeting, so that every member knows who is responsible for what. Assignments balance two goals: using each member's strengths to maintain quality, and giving every member experience across different parts of the project, so that no part of FitFolio depends on a single person. When a member takes on a task in an unfamiliar area, they are paired with a member who has experience in that area and can offer guidance.

8.3 Schedule

The project is scheduled around the course deliverables, each of which is a GitHub milestone. For every deliverable, the team sets a team deadline three days before the course deadline. All work must be merged into `main` by the team deadline, and the remaining three days are reserved for peer review, a quality assurance check of the full deliverable against its checklist and rubric, and final edits. For demonstrations, the team deadline is a full rehearsal of the demonstration, held three days before the first demonstration day. Table 6 lists both deadlines for every deliverable. Unless otherwise stated, all course deadlines are at 11:59 p.m.

Table 6: Team and Course Deadlines

Deliverable	Team Deadline	Course Deadline
Problem Statement, POC Plan, Development Plan	Sep 25, 2026	Sep 28, 2026
Requirements Document and Hazard Analysis Revision 0	Oct 16, 2026	Oct 19, 2026
V&V Plan Revision 0	Oct 30, 2026	Nov 2, 2026
Design Document Revision –1 and CD-Fall Revision 0	Nov 13, 2026	Nov 16, 2026
Proof of Concept Demonstration	Nov 20, 2026	Nov 23 - Dec 4, 2026
Design Document Revision 0	Jan 15, 2027	Jan 18, 2027
Revision 0 Demonstration	Jan 29, 2027	Feb 1 - Feb 12, 2027
V&V Report Revision 0 and CD-Winter Revision 0	Mar 5, 2027	Mar 8, 2027
Final Demonstration (Revision 1)	Mar 12, 2027	Mar 15 - Mar 25, 2027
EXPO Demonstration (poster due)	Mar 28, 2027	Mar 31, 2027
Final Video	Mar 28, 2027	Mar 31, 2027
Final Documentation (Revision 1)	Mar 28, 2027	Mar 31, 2027

To keep work on track between deadlines, each member reports progress and any blockers on their assigned issues during the weekly team meeting. A member who expects to miss a team deadline must tell the team in the group chat at least 24 hours beforehand, so that the task can be supported or reassigned. Feedback on each Revision 0 deliverable will be recorded as issues and addressed throughout the year, rather than left until the Final Documentation.

9 Proof of Concept Demonstration Plan

This section identifies the main risks to the success of FitFolio, describes what the proof of concept (PoC) demonstration will show to convince the team that these risks can be overcome, and outlines what the team will do if the demonstration shows that a risk cannot be overcome as planned.

9.1 Main Risks

FitFolio depends on two external systems that no team member has worked with before. Both are implementation risks: if either cannot be made to work, the app's core loop of earning currency through activity and investing it in real assets cannot be built.

1. **Reading fitness data from the phone’s health platform.** FitFolio’s core promise is that activity is tracked automatically. This requires reading step data through Google Health Connect on Android and Apple HealthKit on iOS, which each have their own Application Programming Interfaces (APIs), permission models, and restrictions on when data can be read. Since FitFolio will be built with React Native, it cannot access these platforms directly, and will depend on third-party libraries to bridge between the app and each platform. These libraries may be incomplete, poorly maintained, or out of date with the latest platform versions, which the team does not control. If step data cannot be read reliably, the app cannot award currency fairly.
2. **Valuing portfolios with real market prices.** The virtual portfolio must follow real asset prices, but market data providers limit how many requests can be made, especially on free plans. If prices cannot be retrieved often enough within these limits, portfolio values will not follow the market closely enough to teach users how investing works.

9.2 Demonstration

The PoC will show one minimal path through FitFolio’s core loop, running on a physical Android phone:

1. **Read the day’s step count** (Risk 1). After the user grants permission, the app reads the current day’s steps from Google Health Connect and displays them. This succeeds if the count matches the phone’s own health app, and updates after the team member walks and reopens the app.
2. **Convert steps into virtual currency** (connects Risks 1 and 2). The step count is converted into currency using a simple fixed rule, and the balance is displayed.
3. **Buy a real asset with virtual currency** (Risk 2). The user spends their balance on shares of one real asset, such as a stock or Exchange-Traded Fund (ETF), at its current market price.
4. **Show the portfolio’s value following the market** (Risk 2). The portfolio’s value is recalculated each time a new price is retrieved. This succeeds if the displayed value matches the asset’s public market price at each update, and the number of requests used shows that the chosen update interval fits within the provider’s free limits.

If time allows, the same steps will also be shown on an iPhone using Apple HealthKit. This is a stretch goal rather than a requirement, so that the core demonstration stays achievable.

The PoC is deliberately not a prototype. It will have a minimal user interface and will not include user accounts, workouts, the social feed, or market scenarios. Its only purpose is to show that the two main risks can be overcome. The team

will rehearse the full demonstration by the team deadline in Table 6, and the demonstrated version will be tagged `poc-demo` on `main`.

9.3 If the PoC Fails

If the demonstration shows that a risk cannot be overcome as planned, the team will adjust the project’s scope rather than its core idea:

- **If step data cannot be read reliably:** The team would first limit syncing to when the user opens the app, rather than in the background, which avoids most platform restrictions. If a third-party library proves unreliable, the team would switch to an alternative library, or write a small amount of platform-specific code to access the health platform directly. If one platform still cannot be supported, FitFolio would support only the platform that works, and the team would discuss with the supervisor whether the second platform should become a future goal. If neither platform works, users would enter their activity manually as a last resort. Since this weakens FitFolio’s promise of automatic tracking, the team would discuss this change in scope with the supervisor before adopting it.
- **If market data limits are too restrictive:** The team would reduce how often prices are updated (e.g., every 15 minutes, or once per day), have a server retrieve each price once and share it across all users rather than requesting it per user, and limit the app to a curated list of assets. If a provider’s limits or cost are still unworkable, the team would switch to a different provider.

9.4 Secondary Risk

A secondary risk is that the market scenarios, such as replays of past downturns, depend on historical price data that may be limited or costly to access. Since this feature is not part of the core loop, it will not be included in the PoC. If historical data proves difficult to obtain, the team would reduce the number of scenarios offered, or build them from a small set of stored historical datasets.

10 Expected Technology

[What programming language or languages do you expect to use? What external libraries? What frameworks? What technologies. Are there major components of the implementation that you expect you will implement, despite the existence of libraries that provide the required functionality. For projects with machine learning, will you use pre-trained models, or be training your own model? —SS]

[The implementation decisions can, and likely will, change over the course of the project. The initial documentation should be written in an abstract way; it should be agnostic of the implementation choices, unless the implementation

choices are project constraints. However, recording our initial thoughts on implementation helps understand the challenge level and feasibility of a project. It may also help with early identification of areas where project members will need to augment their training. —SS]

Topics to discuss include the following:

- Specific programming language
- Specific libraries
- Pre-trained models
- Specific linter tool (if appropriate)
- Specific unit testing framework
- Investigation of code coverage measuring tools
- Specific plans for Continuous Integration (CI), or an explanation that CI is not being done
- Specific performance measuring tools (like Valgrind), if appropriate
- Tools you will likely be using?

[git, GitHub and GitHub projects should be part of your technology. —SS]

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

[What coding standard will you adopt? —SS]

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Appendix — Team Charter

[borrows from [University of Portland Team Charter](#) —SS]

External Goals

[What are your team’s external goals for this project? These are not the goals related to the functionality or quality of the project. These are the goals on what the team wishes to achieve with the project. Potential goals are to win a prize at the Capstone EXPO, or to have something to talk about in interviews, or to get an A+, etc. —SS]

Attendance

Expectations

[What are your team’s expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

Accountability and Teamwork

Quality

[What are your team’s expectations regarding the quality of team members’ preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Attitude

[What are your team’s expectations regarding team members’ ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment with their TA, or the instructor? Are there incentives for reaching targets early? —SS]

Team Building

The team will build cohesion through regular, low-effort rituals rather than occasional large events, since consistent contact matters more than one-off activities.

- **Weekly check-in:** The first five minutes of every weekly team meeting are reserved for an informal check-in, where each member shares how they are doing, both within the project and outside it. This helps the team notice early when a member is overloaded, and adjust workloads before deadlines are affected.
- **Deliverable retrospectives:** After each deliverable is submitted, the team holds a 20-minute retrospective during the next weekly meeting. Each member shares one thing that went well and one thing to improve, and the team agrees on at least one change to try for the next deliverable. This gives every member an equal voice, keeps small frustrations from building up, and feeds directly into each deliverable's reflection.
- **Using our own product:** Once the proof of concept is working, the team will hold a weekly step challenge using FitFolio itself. This builds friendly competition within the team, and gives every member real experience as a user of the product, which will help identify usability issues early.

Decision Making

The team will aim to make decisions by consensus, since decisions that every member agrees with are more likely to be followed through. Decisions are made during the weekly team meeting, or in the group chat for decisions that cannot wait.

- **Voting:** If consensus is not reached after 15 minutes of discussion, the decision is made by majority vote. A vote requires at least four of the five

members to be present or to respond in the group chat, and passes with at least three votes in favour, so that no decision is made by a minority of the team.

- **Area decisions:** Technical decisions within a specific area of the app (e.g., the health data integration or the market engine) are made by the members working on that area, who inform the rest of the team. Any member may ask for such a decision to be brought to a team vote.
- **Scope decisions:** Decisions that change the product's scope or vision, such as removing a feature, are brought to the supervisor before being adopted.
- **Recording decisions:** Every decision and its reasoning is recorded in the relevant meeting issue, so that settled decisions are not revisited without new information.

When disagreements arise, each side will explain their position with a focus on what is best for the project rather than personal preference. If the same disagreement remains unresolved after two consecutive team meetings, it will be raised with the supervisor, who will help the team reach a decision.